

State-feedback control of an inverted pendulum on a cart

J.C. Vaught
EMCH 792, Learning-Based Controls

Objective

The objective of this assignment is to hold the cart-mounted pendulum near its unstable upright equilibrium with state feedback. Mini Project 1 supplied a nonlinear Simulink model of the freely moving cart and pendulum. Here, a horizontal cart force becomes the control input, and the same nonlinear plant tests whether a controller designed from a local linearization can recover from a small release and reject a random disturbance.

Methodology

The state is $z = (x, \dot{x}, \theta, \dot{\theta})^T$, where $\theta = 0$ is upright. The plant retains the Mini Project 1 values $M = 2.0$ kg, $m = 0.5$ kg, $\ell = 1.0$ m, $c_\theta = 0.01$ N m s/rad, and $g = 9.81$ m/s². Linearizing the coupled mass-matrix equations at $z = 0$ gives $\dot{z} = Az + BF$. The controllability matrix has rank four. A linear-quadratic regulator with $Q = \text{diag}(4, 2, 300, 10)$ and $R = 0.5$ sets the feedback $u = -Kz$. The actuator command is limited to $|u| \leq 10$ N, and the nonlinear plant receives $F = u + d$, where d is an external horizontal force on the cart. This choice gives the assignment's base disturbance a precise physical meaning.

The first 12 s test begins at $\theta(0) = 5^\circ$ with $d = 0$. The second begins upright and applies a fixed-seed random force held for 0.1 s per sample, bounded by $|d| \leq 2.5$ N. Simulink integrates the nonlinear model with ode45 and a 0.01 s maximum step. The appendix gives the derivation, model, code, and additional tests.

Results and Discussion

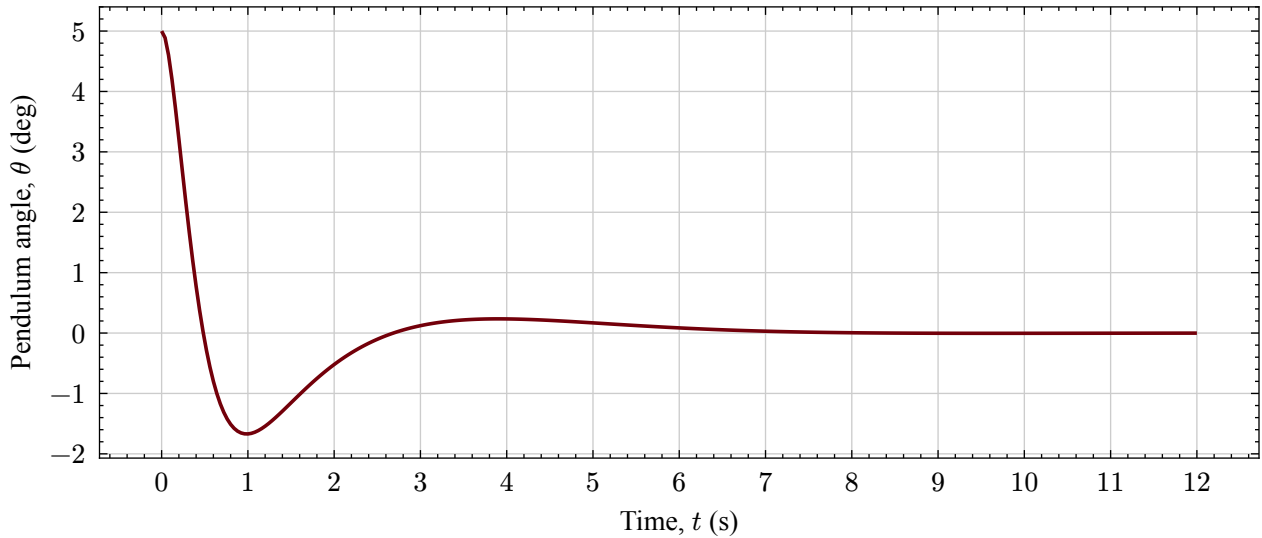


Figure 1: Upright-angle response after a 5° release with no disturbance.

The 5° release returns inside $\pm 1^\circ$ by 1.62 s and stays there. Its peak cart travel is 0.301 m. The response confirms local stabilization of the nonlinear model, including the pivot damping and trigonometric terms absent from the design model.

Results and Discussion (continued)

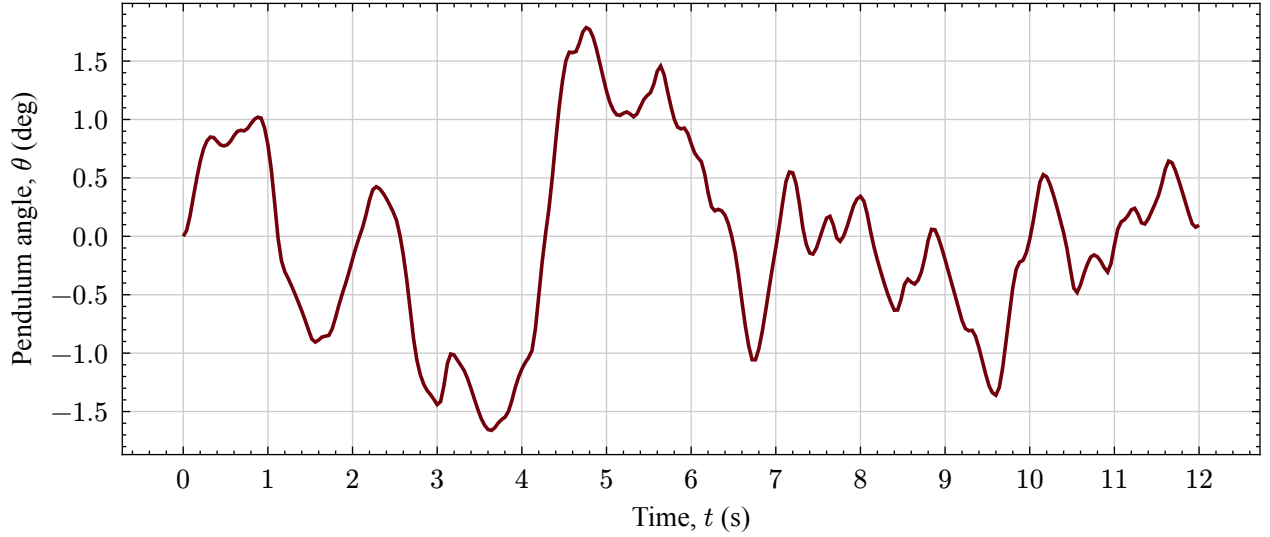


Figure 2: Upright-angle response under the bounded random horizontal disturbance.

With the cart excited throughout the second test, the pendulum's angle has an RMS magnitude of 0.81 degree and a peak of 1.79 degree. The controller therefore keeps the pendulum near upright without requiring the disturbance to stop. The corresponding cart motion and command-force histories appear in the appendix.

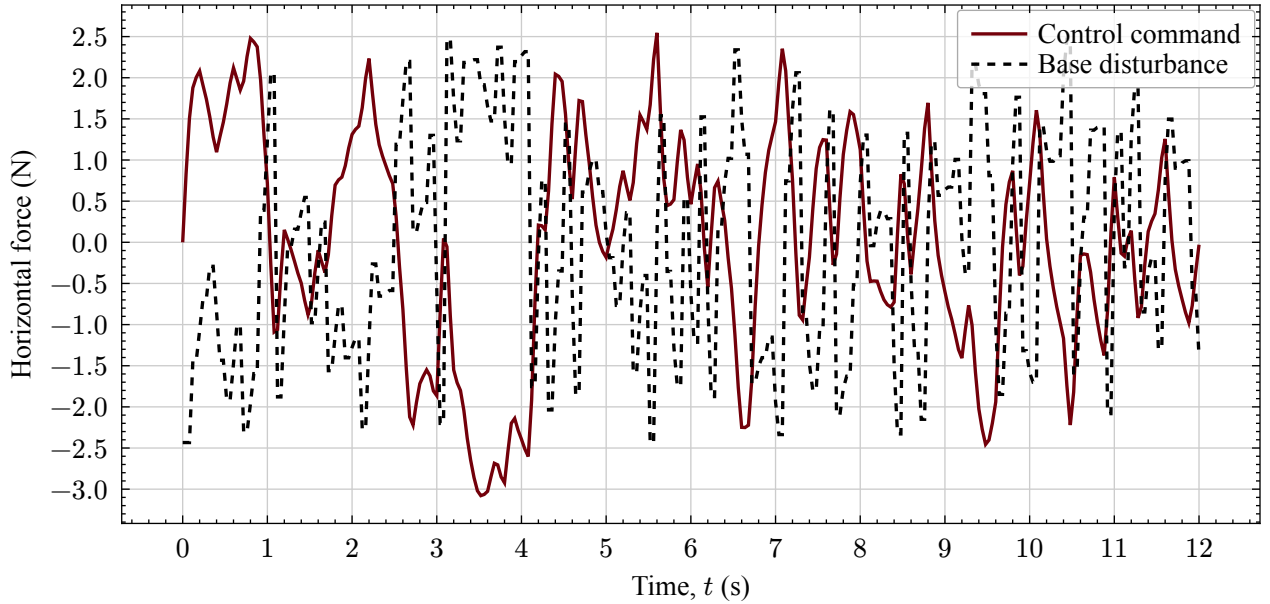


Figure 3: Feedback command and externally applied disturbance in the random-force test. Solid garnet is the command; dashed black is the disturbance.

The actuator never reaches its 10 N limit in the required random test. Under larger perturbations, the 20° release recovers, but the 30° release does not. A 10 N random-force bound also causes sustained saturation and loss of upright control. The local linear design therefore has a finite recovery region. The model assumes an unlimited cart track and exact state measurements; physical rail limits and measurement noise would reduce practical margin.

Appendix A. Nonlinear plant and sign convention

The cart translates in the positive x direction under a positive horizontal force F . The pendulum angle θ measures displacement from the upright position, matching the Mini Project 1 model. The rod is massless, its point mass is m , the cart mass is M , and the pivot has rotational viscous damping c_θ . No track friction or rail stops are present. Applying a force to the cart changes the first of the original two coupled equations to

$$(M + m)\ddot{x} + m\ell \cos(\theta)\ddot{\theta} - m\ell \sin(\theta)\dot{\theta}^2 = F,$$

while the angular equation remains

$$m\ell \cos(\theta)\ddot{x} + m\ell^2\ddot{\theta} + c_\theta\dot{\theta} - mgl \sin(\theta) = 0.$$

The controlled MATLAB Function block solves the resulting 2×2 mass-matrix system at each solver step. The model sends the resulting accelerations through the four inherited integrators. This preserves the Mini Project 1 nonlinear plant and adds only the force input and feedback path.

Parameter	Meaning	Value
M	Cart mass	2.0 kg
m	Pendulum point mass	0.5 kg
ℓ	Pendulum length	1.0 m
c_θ	Pivot rotational damping	0.01 N m s/rad
g	Gravitational acceleration	9.81 m/s ²

The model neglects actuator delay, sensor noise, track friction, and end stops. The reported performance applies to this simulated plant with the stated 10 N command limit.

Appendix B. Upright linearization

For small θ , use $\sin(\theta) \approx \theta$, $\cos(\theta) \approx 1$, and omit the second-order term $\theta\dot{\theta}^2$. At the upright equilibrium, the equations become

$$(M + m)\ddot{x} + m\ell\ddot{\theta} = F,$$

$$m\ell\ddot{x} + m\ell^2\ddot{\theta} + c_\theta\dot{\theta} - mg\ell\theta = 0.$$

Eliminating $\ddot{\theta}$ from the first equation gives

$$\ddot{x} = \frac{F}{M} - \frac{mg}{M}\theta + \frac{c_\theta}{M\ell}\dot{\theta}.$$

Substitution gives the angular acceleration

$$\ddot{\theta} = -\frac{F}{M\ell} + \frac{(M + m)g}{M\ell}\theta - \frac{c_\theta(M + m)}{Mm\ell^2}\dot{\theta}.$$

With $z = (x, \dot{x}, \theta, \dot{\theta})^T$, this is $\dot{z} = Az + BF$. At the assigned parameter values,

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -2.4525 & 0.005 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 12.2625 & -0.025 \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 0.5 \\ 0 \\ -0.5 \end{pmatrix}.$$

The signs of the force entries show that a positive cart force accelerates the cart right and initially accelerates the upright pendulum angle in the opposite direction.

Appendix C. Feedback design

The upright linearization has a controllability matrix $\mathcal{C} = (B, AB, A^2B, A^3B)$ with rank four. The chosen quadratic objective penalizes cart position, cart velocity, upright angle, angular velocity, and control effort through

$$J = \int_0^\infty (z^T Q z + R u^2) dt, \quad Q = \text{diag}(4, 2, 300, 10), \quad R = 0.5.$$

The MATLAB `lqr` calculation returns $u = -Kz$ with $K = (-2.8284, -6.3204, -86.8641, -25.2346)$. The signs reflect the chosen angle and force directions. The nonlinear simulation clips this command to ± 10 N before adding the disturbance. The disturbance itself is an external force, so its magnitude is not counted against the commanded actuator limit.

The theoretical closed-loop poles have negative real parts, but that linear result applies near upright and without saturation. The nonlinear tests below evaluate behavior outside those assumptions. Rebuilding the editable model from `build_controlled_model.m` embeds the computed gain as a numerical block parameter, so the delivered model does not depend on a workspace variable named `K`.

Appendix D. Simulation and measurement protocol

Both required tests run for 12 s with the same continuous nonlinear Simulink model and controller. The first uses $\theta(0) = 5^\circ$, while the remaining initial states and disturbance are zero. The second starts at the upright equilibrium. It applies independent uniform force samples in $[-2.5, 2.5]$ N every 0.1 s and holds each sample constant until the next. The random generator uses seed 79202, and the sampled force is stored in the results file. This is a repeatable test signal, not an unknown future noise process.

The reported peak angle is $\max_t |\theta(t)|$. Root-mean-square angle is $\sqrt{(\frac{1}{N}) \sum_i \theta_i^2}$ on a uniform 0.02 s analysis grid. The 1° settling time is the earliest sampled time after which the angle remains inside that band through the end of the test. Actuator saturation fraction counts grid samples where $|u| \geq 10$ N. A trial is marked recovered only when its final angle and final-two-second RMS angle are both below 2° . Metrics use the full analysis grid; the figure data are thinned to 0.04 s only for the PDF.

The additional tests reuse the same 10 N controller. One sweep starts at 10, 20, 30, 45, and 60° without disturbance. The other scales one fixed random realization to amplitudes of 2.5, 5, 7.5, and 10 N. These sweeps show the consequences of larger releases and disturbance forces without changing the control law.

Appendix E. Release-test signals

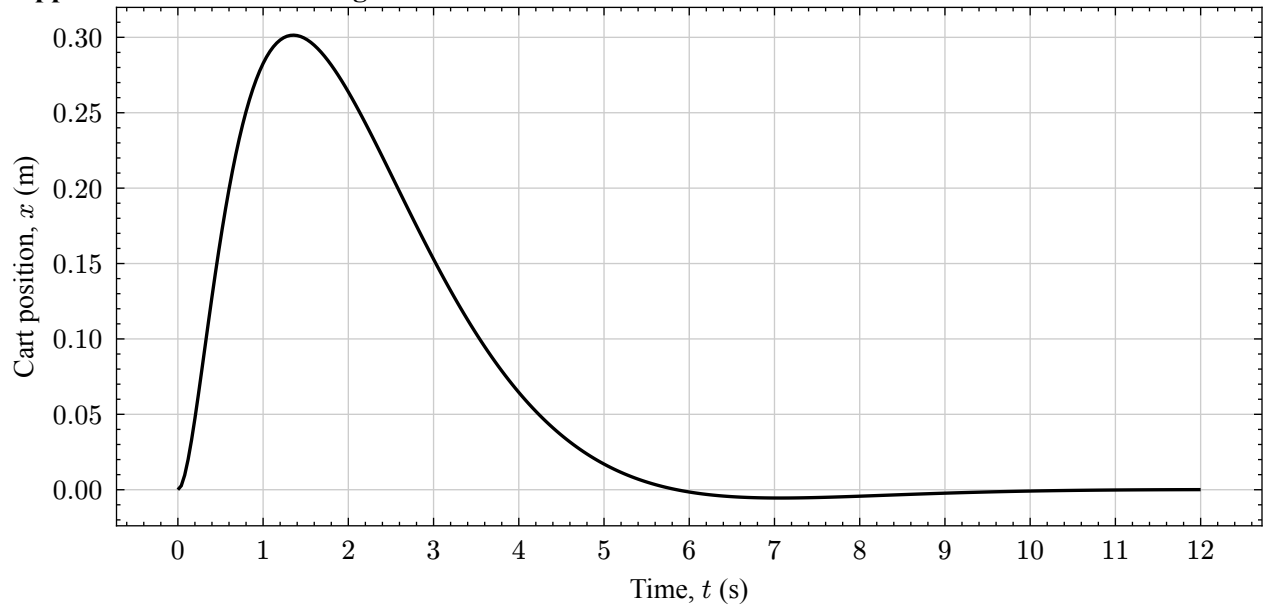


Figure 4: Cart displacement during recovery from the 5° release.

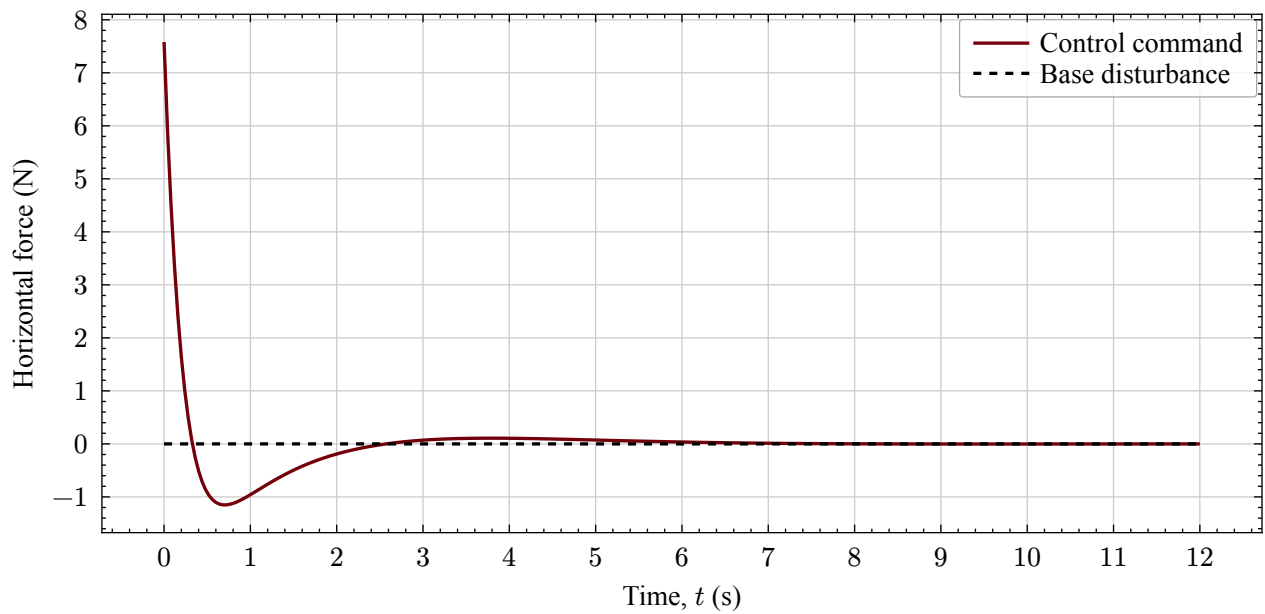


Figure 5: Actuator command during the release test. The disturbance is zero.

The controller shifts the cart to counter the falling moment and then brings the cart toward the origin. Its finite command is represented explicitly by the saturation block in the Simulink model.

Appendix F. Random-disturbance signals

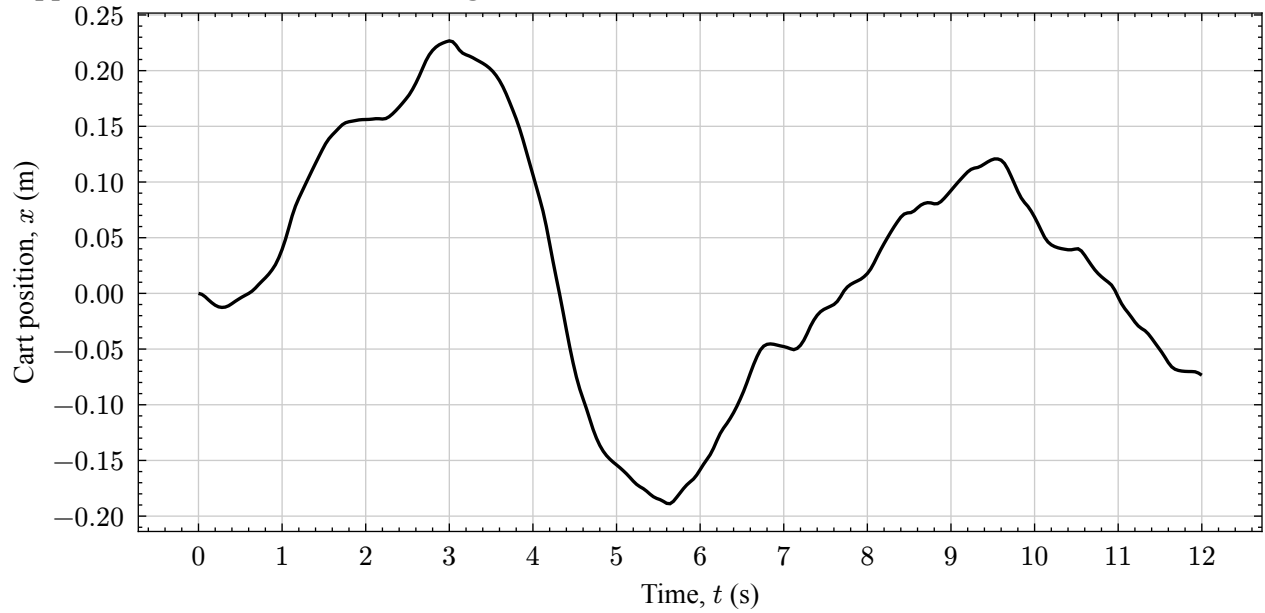


Figure 6: Cart displacement while random force excites the base.

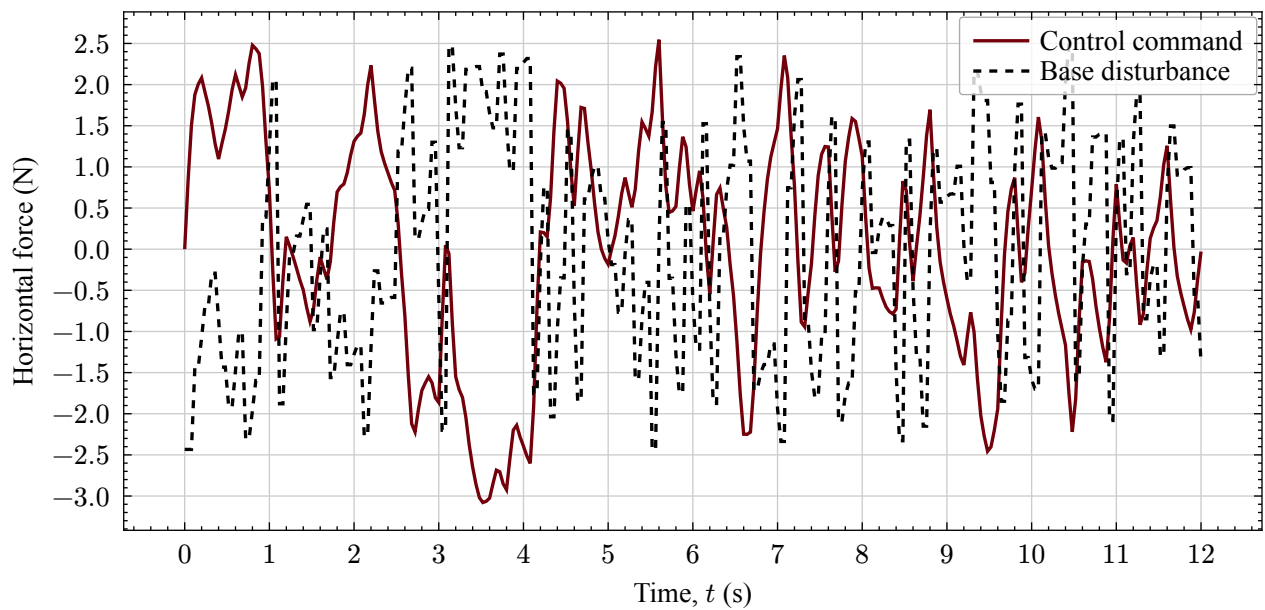


Figure 7: Actuator and disturbance histories for the bounded random-force test.

The cart is free to move along an unlimited track in this model. Its displacement is therefore a separate performance measure from the pendulum angle.

Appendix G. Stress-test summary

The two required tests use 5° initial tilt and a 2.5 N disturbance bound. The following larger cases are diagnostic only. Recovery means that both the final angle and final-two-second RMS angle are under 2 degrees. This criterion does not imply satisfaction of a physical cart-travel constraint.

Initial tilt (deg)	Peak cart travel (m)	Saturation time (%)	Recovered
10	0.646	0.83	Yes
20	2.977	9.82	Yes
30	288.232	100	No
45	287.942	100	No
60	288.324	100	No

Force bound (N)	RMS angle (deg)	Peak angle (deg)	Saturation time (%)
2.5	0.81	1.79	0
5	1.61	3.59	0
7.5	2.42	5.38	0
10	836.17	1866.55	71.88

Because every force-bound case uses the same random draw, the table isolates the effect of scaling the disturbance amplitude. The 20° release recovers only after nearly 3 m of cart travel, while the 30° release saturates throughout and fails. At a 10 N disturbance bound, the angle is unwrapped and records repeated rotations after loss of control. The roughly 288 m cart travel in failed tilt trials is a failure signature of the unlimited-track simulation, not a physically feasible test result.

Appendix H. Editable Simulink model

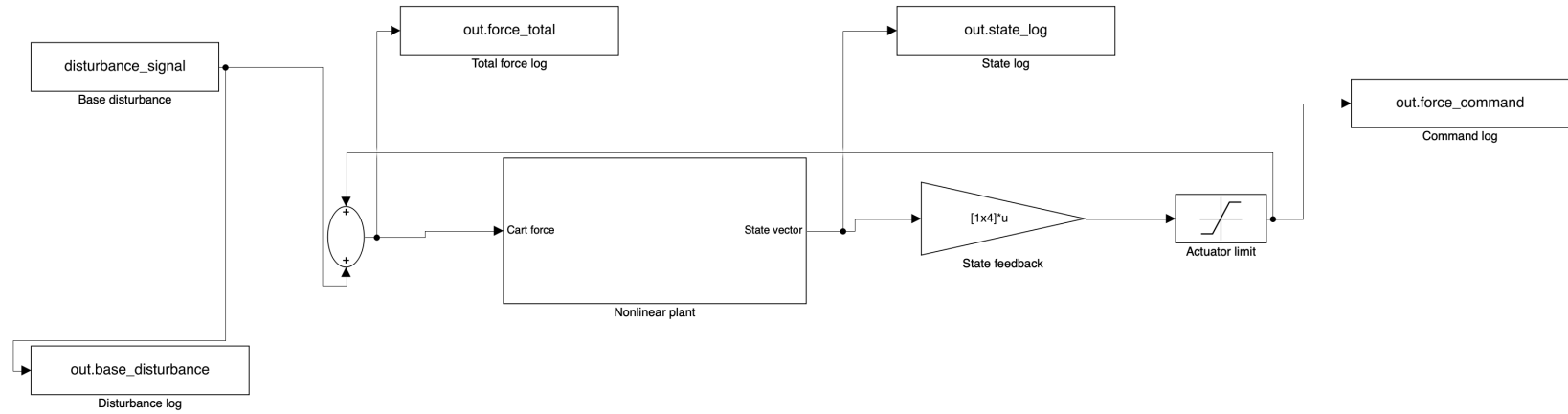


Figure 8: Top-level feedback loop in the delivered `inverted_pendulum_controlled.slx` model.

The top level exposes the disturbance, total cart force, nonlinear plant, feedback gain, actuator limit, and four output blocks that record the simulation histories.

Appendix H. Nonlinear plant detail

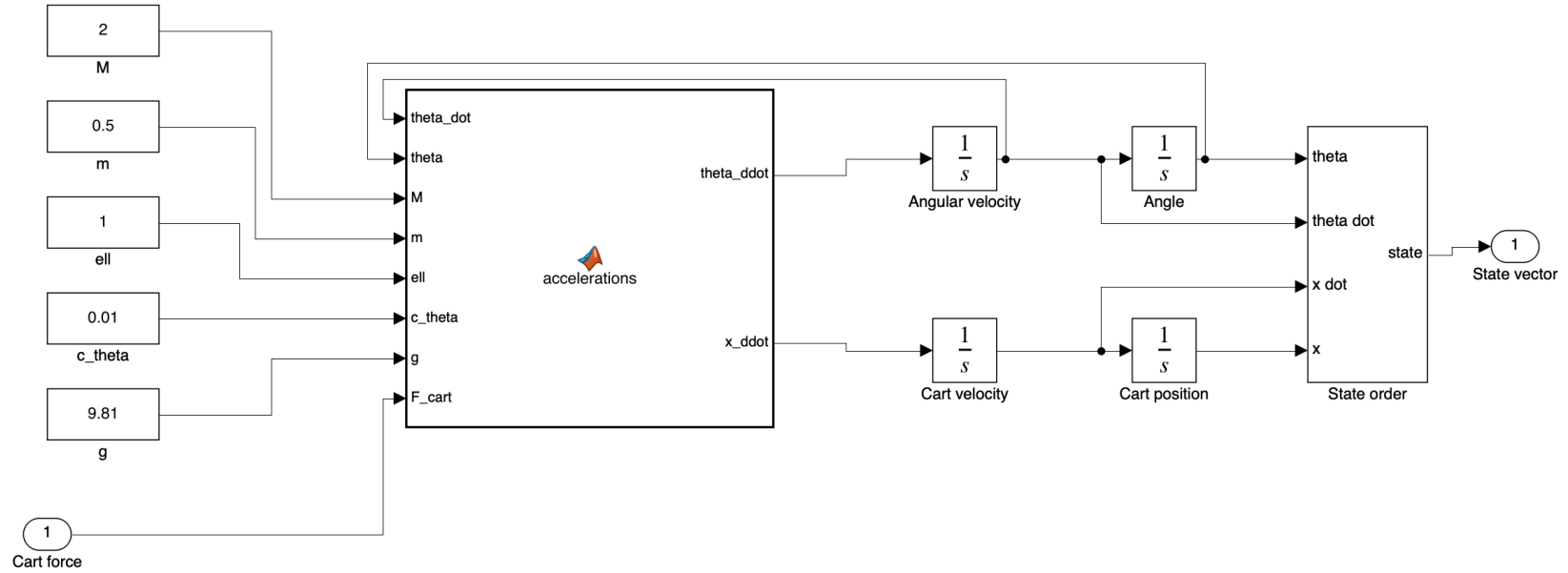


Figure 9: Inside the nonlinear plant subsystem. The equations and four integrators come from Mini Project 1.

Inside the plant, the inherited equations and four integrators retain their original positions. The State order inputs appear from top to bottom as $(\theta, \dot{\theta}, \dot{x}, x)$; its internal wiring assembles the controller vector $(x, \dot{x}, \theta, \dot{\theta})$.

Appendix I. Nonlinear acceleration code

The following is the complete source loaded into the copied MATLAB Function block. The first seven arguments preserve the Mini Project 1 interface; the eighth adds the total cart force.

```
function [theta_ddot, x_ddot] = accelerations(theta_dot, theta, M, m, ell, c_theta, g, F_cart)
% Nonlinear cart-pendulum equations with an applied horizontal cart force.
% Positive theta is the upright perturbation used in Mini Project 1.

s = sin(theta);
c = cos(theta);
mass_matrix = [M + m, m * ell * c; m * ell * c, m * ell^2];
forcing = [F_cart + m * ell * s * theta_dot^2; ...
           m * g * ell * s - c_theta * theta_dot];
acceleration = mass_matrix \ forcing;

x_ddot = acceleration(1);
theta_ddot = acceleration(2);
end
```

Appendix J. Model-construction code

The complete builder copies the Mini Project 1 Simulink model, adds the force input and feedback blocks, and saves an editable Project 2 model.

```
function build_controlled_model(K, force_limit)
% Build a readable controller around the inherited Mini Project 1 plant.
% The nonlinear equations and four integrators are copied into one
% subsystem. Only the feedback loop and logging remain at the top level.

arguments
    K (1, 4) double
    force_limit (1, 1) double {mustBePositive}
end

project_dir = fileparts(mfilename('fullpath'));
previous_dir = pwd;
cleanup = onCleanup(@() cd(previous_dir));
cd(project_dir);

source = 'inverted_pendulum_simulink';
model = 'inverted_pendulum_controlled';
load_system(source);
if bdIsLoaded(model)
    close_system(model, 0);
end
new_system(model);
set_param(model, 'Solver', 'ode45', 'MaxStep', '0.01', ...
            'StopTime', '12', 'SaveTime', 'off', 'SaveOutput', 'off');

plant = [model '/Nonlinear plant'];
add_block('simulink/Ports & Subsystems/Subsystem', plant, ...
          'Position', [460 200 710 320], 'ContentPreviewEnabled', 'off');
delete_line(plant, 'In1/1', 'Out1/1');
set_param([plant '/In1'], 'Name', 'Cart force', ...
          'Position', [30 415 60 435]);
set_param([plant '/Out1'], 'Name', 'State vector', ...
          'Position', [950 235 980 255]);

% Copy the actual Mini Project 1 nonlinear block and its four integrators.
add_block([source '/Nonlinear Equations of Motion'], ...
          [plant '/Nonlinear acceleration'], ...
          'Position', [270 145 500 355]);
add_block([source '/Integrate theta accel'], ...
          [plant '/Angular velocity'], 'Position', [600 170 640 210]);
```

```

add_block([source '/Integrate theta velocity'], ...
    [plant '/Angle'], 'Position', [725 170 765 210]);
add_block([source '/Integrate x acceleration'], ...
    [plant '/Cart velocity'], 'Position', [600 290 640 330]);
add_block([source '/Integrate x velocity'], ...
    [plant '/Cart position'], 'Position', [725 290 765 330]);
set_param([plant '/Angle'], 'InitialCondition', '0');

parameter_names = {'M', 'm', 'ell', 'c_theta', 'g'};
parameter_y = [95 155 215 275 335];
for j = 1:numel(parameter_names)
    name = parameter_names{j};
    add_block([source '/' name], [plant '/' name], ...
        'Position', [45 parameter_y(j) 115 parameter_y(j)+32]);
end
close_system(source, 0);

% The copied MATLAB Function gains an eighth input, the total cart force.
root = sfroot;
chart = root.find('-isa', 'Stateflow.EMChart', ...
    'Path', [plant '/Nonlinear acceleration']);
assert(numel(chart) == 1, 'The copied nonlinear equations were not found.');
```

```

chart.Script = fileread(fullfile(project_dir, 'controlled_accelerations.m'));
set_param([plant '/Nonlinear acceleration'], 'ShowName', 'off');

% State order has the requested external input order theta, theta_dot,
% x_dot, x. Its internal Mux restores the controller state order
% [x; x_dot; theta; theta_dot].
order = [plant '/State order'];
add_block('simulink/Ports & Subsystems/Subsystem', order, ...
    'Position', [835 170 910 330], 'ContentPreviewEnabled', 'off');
delete_line(order, 'In1/1', 'Out1/1');
set_param([order '/In1'], 'Name', 'theta', 'Position', [30 35 60 55]);
set_param([order '/Out1'], 'Name', 'state', 'Position', [260 115 290 135]);
input_names = {'theta dot', 'x dot', 'x'};
input_y = [85 135 185];
for j = 1:3
    add_block('simulink/Sources/In1', [order '/' input_names{j}], ...
        'Position', [30 input_y(j) 60 input_y(j)+20], 'Port', num2str(j+1));
end
add_block('simulink/Signal Routing/Mux', [order '/State vector'], ...
    'Inputs', '4', 'Position', [190 80 195 170]);
connect(order, 'x/1', 'State vector/1');
connect(order, 'x dot/1', 'State vector/2');
connect(order, 'theta/1', 'State vector/3');
connect(order, 'theta dot/1', 'State vector/4');
connect(order, 'State vector/1', 'state/1');
port_labels = sprintf(['port_label(''input'',1,''theta'');' ...
    'port_label(''input'',2,''theta dot'');' ...
    'port_label(''input'',3,''x dot'');' ...
    'port_label(''input'',4,''x'');' ...
    'port_label(''output'',1,''state'');']);
set_param(order, 'Mask', 'on', 'MaskDisplay', port_labels);
connect(plant, 'M/1', 'Nonlinear acceleration/3');
connect(plant, 'm/1', 'Nonlinear acceleration/4');
connect(plant, 'ell/1', 'Nonlinear acceleration/5');
connect(plant, 'c_theta/1', 'Nonlinear acceleration/6');
connect(plant, 'g/1', 'Nonlinear acceleration/7');
connect(plant, 'Cart force/1', 'Nonlinear acceleration/8');
connect(plant, 'Nonlinear acceleration/1', 'Angular velocity/1');
connect(plant, 'Nonlinear acceleration/2', 'Cart velocity/1');
connect(plant, 'Angular velocity/1', 'Angle/1');
connect(plant, 'Cart velocity/1', 'Cart position/1');
connect(plant, 'Angular velocity/1', 'Nonlinear acceleration/1');
connect(plant, 'Angle/1', 'Nonlinear acceleration/2');
```

```

connect(plant, 'Angle/1', 'State order/1');
connect(plant, 'Angular velocity/1', 'State order/2');
connect(plant, 'Cart velocity/1', 'State order/3');
connect(plant, 'Cart position/1', 'State order/4');
connect(plant, 'State order/1', 'State vector/1');

% Restore the original top-level placement and visible logging blocks.
add_block('simulink/Sources/From Workspace', ...
    [model '/Base disturbance'], ...
    'VariableName', 'disturbance_signal', 'Interpolate', 'off', ...
    'OutputAfterFinalValue', 'Holding final value', ...
    'Position', [70 105 225 145]);
add_block('simulink/Math Operations/Sum', [model '/Cart force'], ...
    'Inputs', '++', 'ShowName', 'off', ...
    'Position', [315 240 345 290]);
add_block('simulink/Math Operations/Gain', [model '/State feedback'], ...
    'Gain', mat2str(-K, 16), 'Multiplication', 'Matrix(K*u)', ...
    'Position', [805 220 940 280]);
add_block('simulink/Discontinuities/Saturation', ...
    [model '/Actuator limit'], ...
    'UpperLimit', num2str(force_limit, 17), ...
    'LowerLimit', num2str(-force_limit, 17), ...
    'Position', [1015 230 1090 270]);
log_blocks = {'State log', 'Command log', 'Disturbance log', 'Total force log'};
log_variables = {'state_log', 'force_command', 'base_disturbance', 'force_total'};
log_positions = {[785 75 965 115], [1160 135 1340 175], ...
    [70 355 250 395], [375 75 555 115]};
for j = 1:numel(log_blocks)
    add_block('simulink/Sinks/To Workspace', [model '/' log_blocks{j}], ...
        'VariableName', log_variables{j}, 'SaveFormat', 'Timeseries', ...
        'Position', log_positions{j});
end

connect(model, 'Base disturbance/1', 'Cart force/2');
connect(model, 'Cart force/1', 'Nonlinear plant/1');
connect(model, 'Nonlinear plant/1', 'State feedback/1');
connect(model, 'State feedback/1', 'Actuator limit/1');
connect(model, 'Actuator limit/1', 'Cart force/1');
connect(model, 'Nonlinear plant/1', 'State log/1');
connect(model, 'Actuator limit/1', 'Command log/1');
connect(model, 'Base disturbance/1', 'Disturbance log/1');
connect(model, 'Cart force/1', 'Total force log/1');
set_param(plant, 'ContentPreviewEnabled', 'off');

save_system(model, fullfile(project_dir, [model '.slx']));
close_system(model, 0);
end

function connect(system, source_port, destination_port)
% Auto-routing keeps the editable model aligned after block movement.
add_line(system, source_port, destination_port, 'autorouting', 'on');
end

```

Appendix K. Experiment and verification code

The experiment script derives A and B , checks controllability and closed-loop poles, rebuilds the model, runs all cases, computes metrics, and exports simulation data and the model diagram.

```
function results = run_project2()
% Rebuild and test the nonlinear, force-controlled Mini Project 1 model.
% Run this file from MATLAB. All random inputs use a recorded fixed seed.

project_dir = fileparts(mfilename('fullpath'));
previous_dir = pwd;
cleanup = onCleanup(@() cd(previous_dir));
cd(project_dir);

% The state is z = [cart position; cart velocity; upright angle;
% angular velocity]. Positive F pushes the cart toward positive x.
M = 2.0; m = 0.5; ell = 1.0; c_theta = 0.01; g = 9.81;
force_limit = 10.0;
A = [0 1 0 0; ...
     0 0 -m*g/M c_theta/(M*ell); ...
     0 0 0 1; ...
     0 0 (M+m)*g/(M*ell) -c_theta*(M+m)/(M*m*ell^2)];
B = [0; 1/M; 0; -1/(M*ell)];
Q = diag([4 2 300 10]);
R = 0.5;
assert(rank(ctrb(A, B)) == 4, 'The upright linearization is not controllable.');
```

```
[K, ~, closed_loop_poles] = lqr(A, B, Q, R);
assert(all(real(closed_loop_poles) < 0), 'The designed closed loop is unstable.');
```

```
build_controlled_model(K, force_limit);
model = 'inverted_pendulum_controlled';

% A bounded, zero-order-held horizontal force excites the cart base.
% The same realization is rescaled in the stress test for a fair comparison.
seed = 79202;
rng(seed, 'twister');
stop_time = 12.0;
% Keep one sample beyond the simulation stop so zero-order hold never
% requires extrapolation at the final solver step.
disturbance_time = (0:0.1:(stop_time + 0.1));
unit_random = 2*rand(size(disturbance_time)) - 1;
zero_disturbance = [disturbance_time zeros(size(disturbance_time))];
base_disturbance = [disturbance_time 2.5*unit_random];

perturbation = simulate_case(model, deg2rad(5), zero_disturbance, stop_time);
random_test = simulate_case(model, 0, base_disturbance, stop_time);

% Additional tests show where the finite-force controller loses margin.
angle_grid_deg = [10 20 30 45 60];
angle_stress = repmat(struct(), numel(angle_grid_deg), 1);
for j = 1:numel(angle_grid_deg)
    run = simulate_case(model, deg2rad(angle_grid_deg(j)), ...
        zero_disturbance, stop_time);
    angle_stress(j).initial_angle_deg = angle_grid_deg(j);
    angle_stress(j).metrics = response_metrics(run, force_limit);
end

force_grid_N = [2.5 5 7.5 10];
force_stress = repmat(struct(), numel(force_grid_N), 1);
for j = 1:numel(force_grid_N)
    disturbance = [disturbance_time force_grid_N(j)*unit_random];
    run = simulate_case(model, 0, disturbance, stop_time);
    force_stress(j).amplitude_N = force_grid_N(j);
    force_stress(j).metrics = response_metrics(run, force_limit);
end
```

```

results.parameters = struct('M_kg', M, 'm_kg', m, 'ell_m', ell, ...
    'c_theta_Nms_rad', c_theta, 'g_m_s2', g, 'force_limit_N', force_limit);
results.design = struct('A', A, 'B', B, 'Q', Q, 'R', R, ...
    'K', K, 'poles_real', real(closed_loop_poles), ...
    'poles_imag', imag(closed_loop_poles), ...
    'controllability_rank', rank(ctrb(A, B)));
results.disturbance = struct('seed', seed, 'sample_period_s', 0.1, ...
    'nominal_amplitude_N', 2.5, 'time_s', disturbance_time, ...
    'force_N', base_disturbance(:, 2));
results.perturbation = struct('initial_angle_deg', 5, ...
    'metrics', response_metrics(perturbation, force_limit), ...
    'response', perturbation);
results.random_test = struct('metrics', ...
    response_metrics(random_test, force_limit), 'response', random_test);
results.angle_stress = angle_stress;
results.force_stress = force_stress;

% Retain every computed sample and a compact JSON copy for Typst/Lilaq.
save(fullfile(project_dir, 'simulation_results.mat'), 'results');
plot_data = struct('perturbation', plot_sample(perturbation), ...
    'random_test', plot_sample(random_test), ...
    'disturbance_step_s', 0.1);
write_json(fullfile(project_dir, 'plot_data.json'), plot_data);
summary = rmfield(results, {'perturbation', 'random_test'});
summary.perturbation = results.perturbation.metrics;
summary.random_test = results.random_test.metrics;
write_json(fullfile(project_dir, 'metrics.json'), summary);
fprintf('K = [%s]\n', num2str(K, ' %.6f'));
fprintf('Perturbation settling time = %.3f s; peak angle = %.3f deg.\n', ...
    results.perturbation.metrics.settling_time_s, ...
    results.perturbation.metrics.peak_angle_deg);
fprintf('Random-force RMS angle = %.3f deg; peak angle = %.3f deg.\n', ...
    results.random_test.metrics.rms_angle_deg, ...
    results.random_test.metrics.peak_angle_deg);
disp('Stress test results were written to metrics.json.');
```

```

% Export the actual editable Simulink block diagram for the appendix.
open_system(model);
set_param(model, 'ZoomFactor', 'FitSystem');
print(['-s' model], '-dpng', '-r170', ...
    fullfile(project_dir, 'controlled_model_diagram.png'));
open_system([model '/Nonlinear plant']);
print(['-s' model '/Nonlinear plant'], '-dpng', '-r170', ...
    fullfile(project_dir, 'nonlinear_plant_diagram.png'));
close_system(model, 0);
end
```

```

function response = simulate_case(model, initial_angle_rad, disturbance, stop_time)
% Each simulation uses the same saved model with case-specific inputs.
input = Simulink.SimulationInput(model);
input = input.setVariable('disturbance_signal', disturbance);
input = input.setBlockParameter([model '/Nonlinear plant/Angle'], ...
    'InitialCondition', num2str(initial_angle_rad, 17));
input = input.setModelParameter('StopTime', num2str(stop_time, 17));
output = sim(input);
time = (0:0.02:stop_time)';
response.time_s = time;
state = resample_signal(output.get('state_log'), time);
response.x_m = state(:, 1);
response.x_dot_m_s = state(:, 2);
response.theta_deg = state(:, 3)*180/pi;
response.theta_dot_deg_s = state(:, 4)*180/pi;
response.u_N = resample_signal(output.get('force_command'), time);
response.disturbance_N = resample_signal(output.get('base_disturbance'), time);
response.total_force_N = resample_signal(output.get('force_total'), time);

```

```

assert(all(isfinite(response.theta_deg)), 'The angle response is not finite.');
```

```
end
```

```
function values = resample_signal(signal, time)
```

```
% Remove duplicate solver time stamps before interpolation.
```

```
[logged_time, indices] = unique(double(signal.Time(:)), 'stable');
```

```
logged_values = reshape(double(signal.Data), numel(signal.Time), []);
```

```
values = interp1(logged_time, logged_values(indices, :), ...
```

```
    time, 'linear', 'extrap');
```

```
end
```

```
function metrics = response_metrics(response, force_limit)
```

```
% Report angle control, cart travel, actuator use, and residual error.
```

```
time = response.time_s;
```

```
theta = response.theta_deg;
```

```
u = response.u_N;
```

```
metrics.peak_angle_deg = max(abs(theta));
```

```
metrics.rms_angle_deg = sqrt(mean(theta.^2));
```

```
metrics.final_angle_deg = theta(end);
```

```
metrics.final_2s_rms_angle_deg = sqrt(mean(theta(time >= time(end)-2).^2));
```

```
metrics.peak_cart_position_m = max(abs(response.x_m));
```

```
metrics.final_cart_position_m = response.x_m(end);
```

```
metrics.peak_command_N = max(abs(u));
```

```
metrics.saturation_fraction = mean(abs(u) >= force_limit - 1e-6);
```

```
metrics.control_effort_N2_s = trapz(time, u.^2);
```

```
last_outside = find(abs(theta) > 1, 1, 'last');
```

```
if isempty(last_outside)
```

```
    metrics.settling_time_s = 0;
```

```
elseif last_outside == numel(time)
```

```
    metrics.settling_time_s = NaN;
```

```
else
```

```
    metrics.settling_time_s = time(last_outside + 1);
```

```
end
```

```
metrics.recovered = abs(theta(end)) < 2 && ...
```

```
    metrics.final_2s_rms_angle_deg < 2;
```

```
end
```

```
function sampled = plot_sample(response)
```

```
% Plot data are decimated only after metrics use the full solver response.
```

```
indices = 1:2:numel(response.time_s);
```

```
fields = fieldnames(response);
```

```
for j = 1:numel(fields)
```

```
    sampled.(fields{j}) = response.(fields{j})(indices)';
```

```
end
```

```
end
```

```
function write_json(filename, value)
```

```
fid = fopen(filename, 'w');
```

```
assert(fid > 0, 'Could not open %s.', filename);
```

```
cleanup = onCleanup(@() fclose(fid));
```

```
fprintf(fid, '%s\n', jsonencode(value, 'PrettyPrint', true));
```

```
end
```